

Pacote de Aula Assíncrono (Home-Study): Aula 4

IFPA Campus Ananindeua — Bacharelado em Ciência da Computação (7º Período)

Tema: Virtualização Baseada em Containers (Docker), Ambientes Nuvem e Deploy de Microserviços

Componente Curricular: Computação em Nuvem e Arquitetura Orientada a Serviços (SOA)

Professor: M.Sc. Fábio Ferreira

Formato de Execução: Individual (Atividade Assíncrona / Prática em Casa)

Carga Horária Equivalente: 3 h/a

Mensagem do Professor: A Dinâmica Assíncrona

Olá, pessoal!

Esta é a nossa **Aula 4 (Atividade Assíncrona)**. Caso ocorra qualquer conflito na reserva física do nosso laboratório ou se você preferir otimizar seus estudos, este pacote foi planejado para que você **prepare, configure e valide todo o seu ambiente de desenvolvimento cloud-native diretamente no seu computador pessoal em casa.**

Temos dois grandes objetivos nesta atividade:

Objetivo 1

Dominar os fundamentos conceituais de virtualização
a nível de Sistema Operacional de acordo com o acervo técnico de nossa disciplina.

Objetivo 2

Construir, testar e publicar na internet o seu primeiro
microserviço containerizado.

O projeto prático desenvolvido hoje será a base de infraestrutura que continuaremos a evoluir como atividade prática extraclasses ao longo do bimestre. Leiam o material conceitual abaixo, escolham a sua rota de configuração e executem o passo a passo com atenção. Nos vemos nos canais de comunicação (Teams/E-mail) para dúvidas!

1. Fundamentação Conceitual: A Ciência da Virtualização

Para compreender o que é o **Docker**, você precisa ir além do "comando de terminal" e compreender a arquitetura de sistemas que viabiliza a nuvem.

O Machine Reference Model de Rajkumar Buyya

Conforme estabelecido por **Rajkumar Buyya** no Capítulo 3 do livro *Mastering Cloud Computing*, a arquitetura de computadores moderna possui interfaces de comunicação perfeitamente delimitadas:



Virtualização por Hardware (Hypervisors) vs. Virtualização de SO (Containers)

Virtualização de Hardware (Hypervisors)

No modelo clássico de Virtualização de Hardware (utilizando Hypervisors Tipo 1 como o Xen/ESXi ou Tipo 2 como o VirtualBox/Hyper-V), a virtualização intercepta a camada **ISA**.

- O Hypervisor emula um conjunto completo de instruções de hardware, criando uma máquina virtual inteira.
- Para rodar qualquer aplicação, você é obrigado a instalar um **Guest OS (Sistema Operacional Convidado)** completo dentro da VM.
- **Overhead:** Esse modelo consome gigabytes de RAM e armazenamento, além de criar latência na tradução de instruções de CPU pela camada de emulação.

Virtualização em Nível de SO (Containers / Docker)

No modelo de Virtualização em Nível de Sistema Operacional (Containers / Docker), a virtualização intercepta a camada **ABI**.

- Não há emulação de hardware física e não existe Guest OS.
- Todos os containers compartilham o mesmo Kernel do **Sistema Operacional Hospedeiro** (Host Kernel).
- O Docker opera estritamente no **User Space** (Espaço do Usuário), isolando os processos de cada container diretamente no Kernel usando primitivos lógicos do Linux.

Namespaces: Isolamento de visualização. Garante que o container tenha seu próprio sistema de arquivos, sua própria pilha de rede (portas lógicas) e sua própria árvore de processos (PID), impedindo-o de enxergar o host ou outros containers.

2. Preparando o Seu Ambiente (Duas Rotas Possíveis)

Escolha UMA das duas rotas de configuração de ambiente abaixo para trabalhar em sua casa:

ROTA A (Recomendada): GitHub Codespaces (100% Online e Leve)

Se o seu computador pessoal possui especificações modestas (ex: processador Celeron, Core i3 antigo, ou menos de 8 GB de RAM), esta rota é perfeita. Toda a execução do Docker ocorrerá nos datacenters da Microsoft, de forma gratuita, controlada pelo seu navegador.

1. Acesse github.com e faça login na sua conta pessoal.
2. Acesse o repositório que criará no Passo 1 do Roteiro Prático.
3. Clique no botão verde <> Code, selecione a aba Codespaces e clique em Create codespace on main.
4. O GitHub abrirá uma versão completa do editor VS

Code no seu navegador, rodando em um servidor Linux de alta velocidade com o Docker pré-instalado e

ativo por padrão.

ROTA B (Avançada/Local): Instalação do Docker Desktop Local (Modo Per-User)

Se você possui um notebook de melhor desempenho (mínimo de 8 GB de RAM) e quer aprender a rodar o Docker diretamente na sua máquina, siga este passo a passo.

Nota de segurança: Se o computador que você usa possui restrições de permissões ou se você não possui senha de administrador no Windows, a instalação em modo "Per-User" (Por Usuário) permite rodar o Docker de forma isolada no seu perfil.

Pré-requisitos de Hardware e SO no seu PC:

- Processador de 64 bits com virtualização ativa na placa-mãe.
- **Ativar Virtualização na BIOS:** Ao reiniciar o seu computador, acesse a BIOS (geralmente pressionando F2, F12 ou Del) e garanta que as opções Intel



Passo a Passo da Instalação Local (Rota B)

01

Baixar o Instalador

Acesse a documentação oficial do Docker e baixe o instalador do Docker Desktop para Windows.

02

Executar em Modo Usuário Comum

Dê um duplo clique no instalador. Se o Windows abrir a janela azul do UAC solicitando senha de administrador, clique em **Não** ou ignore. O instalador detectará que você deseja instalar apenas para o seu usuário (instalação *Per-User*) e prosseguirá gravando os arquivos na sua pasta de usuário (AppData\Local).

03

Ativar o backend do WSL 2

Garanta que a caixa "Use the WSL 2 based engine" esteja marcada durante a instalação.

04

Instalação Automática do WSL 2

Se o Windows reclamar que o WSL 2 não está instalado, abra o seu CMD e execute:

```
wsl --install
```

Se o Windows solicitar elevação, permita para que ele ative os recursos de máquina virtual e atualize o kernel do Linux.

01

02

03

3. Roteiro Prático: Construindo a API REST e o Container

Agora que o seu ambiente está configurado (via Codespaces ou Local), vamos ao desenvolvimento e empacotamento da nossa aplicação de engenharia.

PASSO 1: Inicializando o Repositório Git

1. Vá até o seu painel do GitHub e crie um novo repositório chamado `ifpa-nuvem-pratica1-seu-nome`.
2. Configure-o como **Public** (obrigatório para que a nuvem consiga ler o seu código no deploy).
3. Marque a caixa de seleção **"Add a README file"**.
4. No campo "Add .gitignore", escolha o template para **Python**.
5. Clique em **Create repository**.

PASSO 2: Criando os Arquivos de Código-Fonte

No seu editor (VS Code Web do Codespaces ou VS Code Local conectado ao seu Docker), crie a estrutura de arquivos a seguir na raiz do seu projeto:

Arquivo 1: app.py (O Microserviço Flask)

Escreva o código abaixo no arquivo `app.py`. Preste atenção nas orientações de trilha o

```
from flask import Flask, jsonify, request
import os
import datetime
```

```
app = Flask(__name__)
```

```
# Banco de dados simulado em memória (Segregação lógica Multitenant)
pedidos_armazenados = []
```

```
@app.route('/', methods=['GET'])
```

```
def healthcheck():
```

```
    return jsonify({
```

```
        "status": "Online",
```

```
        "timestamp": datetime.datetime.now().isoformat(),
```

```
        "ambiente": "Container Linux (Docker)",
```

```
        "disciplina": "Computacao em Nuvem e SOA - IFPA",
```

```
        "api_endpoint": "/predict [POST]"
```

```
    }), 200
```

```
# =====
```

```
# SELEÇÃO DA TRILHA DE CÓDIGO (Escolha e use APENAS UMA das duas abaixo)
```

```
# =====
```

```
# --- TRILHA INTEGRADA (Alunos de Tópicos Avançados I) ---
```

```
# Se você cursa as duas disciplinas, utilize esta lógica:
```

```
@app.route('/predict', methods=['POST'])
```

```
def predict_mock_ia():
```

```
    data = request.get_json() or {}
```

```
    user_prompt = data.get("prompt")
```

```
    tenant_id = data.get("tenant_id" "default_tenant")
```

Arquivos 2, 3 e 4: requirements.txt .dockerignore

Arquivo 2: requirements.txt (Gerenciador d

Crie o arquivo requirements.txt para declarar as dependências que o Python precisará

```
Flask==3.0.2  
gunicorn==21.2.0
```

Arquivo 3: Dockerfile (Mecanismo de Cons

Crie o arquivo Dockerfile (sem extensão). Usaremos uma imagem base ultra-leve com drasticamente o tamanho do container e economizar memória no seu PC:

```
# Passo 1: Imagem base minimalista de Python com Alpine Linux  
FROM python:3.9-alpine  
  
# Passo 2: Estabelecer o diretório de trabalho isolado dentro do container  
WORKDIR /app  
  
# Passo 3: Copiar o requirements e instalar dependências sem salvar cache em disco  
COPY requirements.txt requirements.txt  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Passo 4: Copiar o restante do código fonte do microserviço  
COPY app.py app.py  
  
# Passo 5: Expor a porta lógica do container  
EXPOSE 5000  
  
# Passo 6: Comando padrão iniciando a API via Gunicorn WSGI Server em nível de pro  
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

PASSO 3: Executando Comandos Docker no Terminal

No terminal do seu Codespaces ou no terminal do seu Windows (com o Docker Desktop) siga as instruções:

1

Build da Imagem Docker

```
docker build -t app-nuvem-ifpa:v1 .
```

Observe o processo do terminal fatiando a construção em camadas. Como o Docker é ultra-rápido.

2

Listar as Imagens Construídas

```
docker images
```

Certifique-se de que a imagem `app-nuvem-ifpa` com a tag `v1` está listada e salva no arquivo final em disco.

3

Executar o Container Mapeando as Portas de Rede:

```
docker run -p 5000:5000 app-nuvem-ifpa:v1
```

O parâmetro `-p 5000:5000` diz ao Docker para escutar a porta física do seu sistema operacional e repassar todas as conexões de entrada de forma imediata para a porta do container Linux.

Testando a Rota GET (Healthcheck):

PASSO 4 e 5: Commit, Push e Deploy Pública

PASSO 4: Commit e Push para o seu Repositório

Uma vez validado o funcionamento das rotas, retorne ao terminal onde o container está rodando e digite `C` para desligar o serviço de forma segura. Envie as atualizações para o seu repositório público.

```
git add .  
git commit -m "feat: api rest flask com dockerfile alpine homologada"  
git push origin main
```

PASSO 5: Realizando o Deploy na Nuvem Pública (Render)

Com o código fonte da sua API de engenharia seguro e controlado por versão, delegando o provisionamento de hardware, a montagem do container em produção e a exposição pública criptografada:

1. Acesse **dashboard.render.com** e faça login com a sua conta do **GitHub**.
2. Clique no botão azul **New** (canto superior direito) e selecione a opção **Web Service**.
3. Em **Connect a repository**, procure pelo seu repositório `ifpa-nuvem-pratica1-seu-nome`.
4. Defina os seguintes parâmetros obrigatórios de provisionamento na Render:
 - **Name:** `app-nuvem-ifpa-seu-nome` (utilize o seu nome para evitar conflito de nomes).
 - **Region:** Selecione a região padrão recomendada (ex: Oregon - US).
 - **Branch:** `main`
 - **Runtime / Language:** Escolha a opção **Docker** (isso é fundamental! Ao selecionar a opção Docker, a Render a ignorar compiladores genéricos e construir todo o ambiente de desenvolvimento a partir das instruções de compilação escritas no seu Dockerfile).
5. Role até a parte inferior, certifique-se de que o plano selecionado é o **Free (\$0/mo)**.
6. Acompanhe o log de compilação em tempo real. A Render fará o download do seu código e criará o container Linux e publicará o serviço.



4. Estudo Dirigido e Autoavaliação (Teórico-Prática)

Para fixar os conceitos científicos e teóricos abordados na prática e nos preparar para as discussões em sala de aula e avaliações do ENADE, resolva as questões abaixo em seu relatório:

Questões sobre Fundamentação (Buyya / Erl)

1

Com base na taxonomia do **Machine Reference Model** de **Rajkumar Buyya**, explique de forma detalhada qual a diferença física e operacional de escopo de execução entre um software executado em uma Máquina Virtual convencional (como no VirtualBox ou VMWare) e um software executado sob uma imagem de Container Docker. Onde cada um intercepta as interfaces do sistema?

2

Por que o uso de uma imagem base contendo o **Alpine Linux** no Dockerfile (FROM python:3.9-alpine) é considerado uma prática altamente eficiente em arquiteturas de nuvem? Conecte sua resposta com os conceitos de *overhead computacional*, *armazenamento* e *segurança lógica*.



Conexão ENADE (Questões de

Questão A (Adaptada do ENADE 2021 – Tec e Desenvolvimento de Sistemas)

Em arquiteturas de computação em nuvem, o modelo de serviços PaaS (Plataforma de Responsabilidade Compartilhada entre o cliente e o provedor de nuvem pública. Como acabou de realizar ao implantar seu serviço na plataforma Render, assinale a alternativa correta sobre a divisão de responsabilidades técnica:

(A)

O cliente é o responsável por atualizar os patches de segurança do sistema operacional do host e o provedor gerencia as rotas internas da API.

(B)

O provedor de nuvem é responsável pelo provisionamento físico dos servidores, dos hypervisors e o sistema operacional. O cliente responsabiliza-se por gerenciar as dependências de runtime e o código da aplicação.

(C)

O cliente deve provisionar os roteadores físicos no datacenter da Render para garantir a elasticidade elástica da API.

(D)

A plataforma PaaS tem a responsabilidade de segurança e infraestrutura, sendo o cliente apenas como locatário.

Questão B (Adaptada do ENADE 2021 – Bacharelado em Ciência da Computação)

"A virtualização de sistema operacional utiliza isolamento lógico provido por recursos como namespaces e cgroups para segmentar e isolar processos concorrentes compartilhando o mesmo hardware. No Docker, os containers são criados utilizando primitivos lógicos nativos e fundamentais do Kernel Linux utilizados pelo Docker para criar namespaces para isolamento visual de processos/portas de rede e a limitação física de uso de CPU/RAM."

(A)

(B)



5. Diretrizes de Entrega

Cada aluno deverá elaborar um Relatório Técnico em formato PDF contendo:

1

Identificação

Nome completo, Matrícula, Turma (7º BCC) e Data.

2

Dados de Acesso

Link do seu repositório público do GitHub e a URL pública HTTPS ativa gerada pela Render.

3

Prints de Evidência

- Print do console do terminal do Codespaces ou local listando o tamanho da imagem compilada (docker images).
- Print do log de build bem-sucedido no painel do console da Render.
- Prints dos dois testes funcionais das rotas (GET na rota raiz e POST no endpoint /predict com status de sucesso) rodando na URL pública da Render.

4

Respostas do Estudo Dirigido

As resoluções completas e justificadas das 4 questões teóricas contidas na Seção 4 deste documento.



Prazo de Entrega: Envie o arquivo PDF do relatório e o arquivo .zip contendo os códigos (app.py, requirements.txt, Dockerfile e .dockerignore) na tarefa aberta no **SIGAA / Teams** até o dia **03/Setembro (Próxima quinta-feira)**, servindo como registro da sua presença assíncrona e nota complementar de laboratório.